```
profit = (salePrice) ->
  overhead = 140
  costPrice = 100
  numberSold = (salePrice) ->
    50 + 20/10 * (200 - salePrice)
  revenue = (salePrice) ->
    (numberSold salePrice) * salePrice
  cost = (salePrice) ->
    overhead + (numberSold salePrice) * costPrice

  (revenue salePrice) - (cost salePrice)
```

These variables are scoped inside the profit function.

It's easier to comprehend a program with good encapsulation. A program with poor encapsulation is a bit like emptying all the boxes, jars, and packets from your pantry into one big bucket. Good for making soup, but not much good for anything else.

Before continuing, it's time for an important lesson about parentheses.

6.1.2. Where arguments need parentheses

Remember that when you're invoking a function with arguments, parentheses are optional:

```
revenue salePrice
revenue(salePrice)
```

These are effectively the same. The parentheses aren't required.

But parentheses *are* sometimes necessary to indicate where the arguments end. Why? Well, how do you think this expression is evaluated?

```
revenue salePrice – cost salePrice
```

To your eyes this example probably looks like it subtracts the cost at the sale price from the revenue at the sale price. But that's not how the CoffeeScript compiler sees it, and the JavaScript you get might not be what you expect:

```
revenue(salePrice(-(cost(salePrice))));
```